

Grouping and Storing Data

Python and its libraries provide many ways to group data together. Some important ones:

- Lists, Tuples, Sets, Dictionaries (built-in to Python)
- Arrays (found in the NumPy library)
- DataFrames (found in the Pandas library)

The above are listed in order of increasing functionality and sophistication

In general you should use the simplest one that meets your needs...

Sequences (built into Python)

Sequences are a basic type of structure used to group data. You don't need to import any additional libraries to use them.

We will focus mostly on one type of sequence: **lists**! (**sets** and **tuples** are other useful types of sequences that you may want to investigate on your own)

A **list** is an ordered sequence of values that can be changed.

You can create a list by using square brackets "[]"

Let's create a list

```
In [1]: #each element is separated by commas

School_locations = ["California", "Illinois", "North Carolina", "Texas",
"Georgia", "Washington D.C."]
School_locations
```

```
Out[1]: ['California',
'Illinois',
'North Carolina',
'Texas',
'Georgia',
'Washington D.C.']
```

Lists can have different data types!

```
In [2]: Random_stuff = ["California", 38, 3.14159] #A string, int, float
Random_stuff
```

```
Out[2]: ['California', 38, 3.14159]
```

Built-in Functions

Recall Python has built-in functions of the form `function_name()`. A very common one is the `print()` function.

`type()` is another important function. We can call this on a number, string, list, or any object to see its type!

```
In [3]: print("California")
print(type("California"))
print(type(38))
print(type("38"))
print(type(3.14))
print(type(Random_stuff))
```

```
California
<class 'str'>
<class 'int'>
<class 'str'>
<class 'float'>
<class 'list'>
```

We can also find the length of a list in the same way we found the length of a string:

```
In [4]: len(Random_stuff)
```

```
Out[4]: 3
```

Note: We will discuss more built-in functions as they become important.

Accessing values from a list

Each element of a sequence is assigned an index corresponding to its position where **indices start at 0**. We can access an element by calling the sequence or list and putting in square brackets the element we want!

```
In [5]: School_locations = ["California", "Illinois", "North Carolina", "Texas",
"Georgia", "Washington D.C."]
```

```
In [6]: School_locations[1]
```

```
Out[6]: 'Illinois'
```

```
In [7]: School_locations[0]
```

```
Out[7]: 'California'
```

Negative indexes count from the end of the sequence

```
In [8]: School_locations[-1]
```

```
Out[8]: 'Washington D.C.'
```

This is useful if our list is long or we want the first or second element from the end!



Code Comprehension:

There are six elements in our list `School_locations`, what will happen when we run the following code?

```
School_locations = ["California", "Illinois", "North Carolina",  
"Texas", "Georgia", "Washington D.C."]
```

```
School_locations[6]
```

This raises an `IndexError`. The list has six elements at indexes 0 through 5, so index 6 is out of range.

List Slicing

We can also extract a "slice" of a list.

The range of elements can be specified with a colon. The output is a list starting at left index and stopping at (right index - 1). The range you specify is a half-closed interval [start,end)

```
list[start: end]
```

```
In [9]: School_locations[1:3] # up to but not including the end of the slice
```

```
Out[9]: ['Illinois', 'North Carolina']
```

The above prints out the second and third elements.

We can still slice a list by 'leaving out' a starting (or ending) position. The missing position will revert to the default. The default start value is the beginning of the list. The default end value is the end of the list.

```
In [10]: School_locations[:3] #start at beginning stop at index end-1 -----> 3-1=2
```

```
Out[10]: ['California', 'Illinois', 'North Carolina']
```



Code Comprehension:

What will the following output?

```
School_locations = ["California", "Illinois", "North Carolina",  
"Texas", "Georgia", "Washington D.C."]
```

```
School_locations[:]
```

A. `[]`

B. `["California", "Washington D.C."]`

C. `["California", "Illinois", "North Carolina", "Texas",
"Georgia", "Washington D.C."]`

Default Settings: List slicing

It might be useful to take out every even indexed element from a list.

There is an optional argument we can use when slicing a list, the step. In general we can slice by using `list[start: end: step]` where the default (what is used when it isn't specified) of step is 1.

```
In [11]: School_locations = ["California", "Illinois", "North Carolina", "Texas",  
"Georgia", "Washington D.C."]  
  
School_locations[::2]
```

```
Out[11]: ['California', 'North Carolina', 'Georgia']
```

What do you think the following will output?

```
School_locations[:3:3]
```

Operations and manipulation on lists

- We can insert items into lists!
 - at either end
 - in the middle
- Count how many items have a specific value
- Sort elements

How do we do this? By using methods. We already saw string methods! These can be extended to lists too.

Methods are particular built-in functions that work on objects in Python. There are specific methods that work for all *list* objects!

Methods take the form

```
list.method()
```

Built-in functions can also be applied to objects in Python. Recall they take the form

```
function_name(list)
```

Appending to a list

```
In [12]: #you can append an item to the end of a list

School_locations = ["California", "Illinois", "North Carolina", "Texas",
                    "Georgia", "Washington D.C."]

School_locations.append('Michigan')
School_locations
```

```
Out[12]: ['California',
          'Illinois',
          'North Carolina',
          'Texas',
          'Georgia',
          'Washington D.C.',
          'Michigan']
```

```
In [13]: #Or insert a value at a particular index
School_locations.insert(1,'tomato')
School_locations
```

```
Out[13]: ['California',
          'Illinois',
          'North Carolina',
          'Texas',
          'tomato',
          'Georgia',
          'Washington D.C.',
          'Michigan']
```

Note we didn't assign `School_locations`, but this automatically changes the object.

Let's make a copy of our list so we can see how this works.

```
In [14]: School_location_og = School_locations.copy()
School_location_og
```

```
Out[14]: ['California',  
          'Illinois',  
          'North Carolina',  
          'Texas',  
          'tomato',  
          'Georgia',  
          'Washington D.C.',  
          'Michigan']
```

We can append something else to our list. Note, even though the list contains only strings, I can append an integer as lists accept any data type.

```
In [15]: School_locations.append(2023)  
School_locations
```

```
Out[15]: ['California',  
          'Illinois',  
          'North Carolina',  
          'Texas',  
          'tomato',  
          'Georgia',  
          'Washington D.C.',  
          'Michigan',  
          2023]
```

This does not change our original copy of the list.

```
In [16]: School_location_og
```

```
Out[16]: ['California',  
          'Illinois',  
          'North Carolina',  
          'Texas',  
          'tomato',  
          'Georgia',  
          'Washington D.C.',  
          'Michigan']
```

How do we find methods?

- Use online documentation!

<https://docs.python.org/3/tutorial/datastructures.html>

- Use built-in function `dir()`

```
In [17]: print(dir(School_locations))  
  
print(dir(list))
```

```
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
['__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__ge
tattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd_
__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__le
n__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex_
__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__
sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'coun
t', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__',
['__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__ge
tattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd_
__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__le
n__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex_
__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__
sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'coun
t', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

A common built-in function:

```
In [18]: len(School_locations)
```

```
Out[18]: 9
```



Exercise:

Create a list with at least 4 items. (Recall items can be a string, int, float)

With this list, perform the following:

```
In [19]: my_list = ["apple", "banana", 7, 3.5, "cherry"]
my_list
```

```
Out[19]: ['apple', 'banana', 7, 3.5, 'cherry']
```

1. Remove elements 2 and 3

```
In [20]: del my_list[2:4]
my_list
```

```
Out[20]: ['apple', 'banana', 'cherry']
```

2. Append your favorite number to the end of the list

```
In [21]: my_list.append(7)
my_list
```

```
Out[21]: ['apple', 'banana', 'cherry', 7]
```

3. Insert the string "Math is fun" at index 2

```
In [22]: my_list.insert(2, "Math is fun")
         my_list
```

```
Out[22]: ['apple', 'banana', 'Math is fun', 'cherry', 7]
```

4. Did your above operations change your original list, or create a copy?

My operations above changed my original list.

5. (optional) Given list `number_list = [1,2,3,4,5,6,7]` , create a new list `new_list = [6,4,2]` in 2 lines of code. You may have to use list methods.

```
In [23]: number_list = [1, 2, 3, 4, 5, 6, 7]
         new_list = number_list[1:6:2]
         new_list.reverse()
         new_list
```

```
Out[23]: [6, 4, 2]
```

Grouping Data using Arrays

Arrays (from the NumPy library) are another way to collect data. Like a list, they contain a sequence of values. But, unlike a list, all elements of an array must have the same data type. This is because NumPy arrays were built for efficient computation. They can perform operations on all elements in one step. They can also do elementwise computation. This means that, for example, if you add two arrays together, the result will be an array where the element at each index is the sum of the elements at the index in the two original arrays. For this reason, when two arrays are added (or subtracted, multiplied, or divided) they must have the same size.

Lists vs Arrays

- Lists are more flexible
 - Can contain elements of different types
- NumPy arrays have some advantages
 - size - they take up less computer memory than lists
 - performance - faster access than lists
 - functionality - linear algebra functions built-in
 - can be multiple dimensions

Creating an array

First import the NumPy library. We do this using an import statement and we give NumPy the alias np. This makes it so that any time we want to access the NumPy library, we only need to type np.

We can create an array using `np.array()` which takes in a list of values of the same data type. Below is an example of writing the import statement and creating an empty array. The array is empty because the list has no elements.

```
import numpy as np

np.array([])
```

(If you get an error when running the previous cell, return to the "Installation Instructions" document and ensure that you have properly installed NumPy.)

Now, let's actually import numpy and create an array with two elements.

```
In [24]: import numpy as np

np.array([1,2])
```

```
Out[24]: array([1, 2])
```

Below, we create a list that contains integers and a string.

```
In [25]: tomato_list = [22, 38, 26, 35, 35, 'tomato']
print(tomato_list)

[22, 38, 26, 35, 35, 'tomato']
```

Let's see what happens when we try to make an array from this list.

```
In [26]: tomato_array = np.array([22, 38, 26, 35, 35, 'tomato'])
print(tomato_array)

['22' '38' '26' '35' '35' 'tomato']
```

A list can have different types, but an array will default to one. So you can see all ints were changed to strings (notice the single quotes around the numbers).

An array will make sure everything is the same type.

Exploring arrays

```
In [27]: #create an array - converting a Python list to a numpy array
prime50_array = np.array([2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
43, 47])

print(type(prime50_array))
```

```
<class 'numpy.ndarray'>
```

Extraction and slicing of one dimensional arrays work exactly the same as lists.

```
In [28]: print(prime50_array[1]) #extracts second element  
prime50_array[1:2] #starting at index 1 and up to (but excluding) index 2
```

```
3
```

```
Out[28]: array([3])
```



Code Comprehension:

Will the following give the same or different outputs?

```
prime50_array[1::2]
```

```
prime50_array[:,::2]
```

Your response here.

Arrays have attributes

Attributes are characteristics of an object. We can view an object's attributes by using the dot operator `.` similarly to when we used methods by using the syntax `object.method()`. However, to access an attribute, we don't use parentheses. (See <https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html>)

Two commonly used attributes of arrays are size, which we can access using `.size`, and shape, accessed using `.shape`.

```
In [29]: prime50_array.size
```

```
Out[29]: 15
```

```
In [30]: prime50_array.shape
```

```
Out[30]: (15,)
```

Arrays have useful methods

Like lists, there are a lot of useful methods we can use on array objects. Here are a couple:

- `.sum()`
- `.mean()`

- `.nonzero()`

(see the documentation for a complete list:

<https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html>)

```
In [31]: print(prime50_array.sum())
#We cannot always sum a list because a list can have different data types!
```

328

NumPy also has many useful built-in functions to use on arrays!

Notice there is often more than one way to do a common operation!

```
In [32]: print(prime50_array.sum()) #sum method

print(np.sum(prime50_array)) #sum function (from NumPy library)

print(np.count_nonzero(np.array([1,2,0,2,1,0,2])))

print(prime50_array.mean()) #mean method

print(np.mean(prime50_array)) #mean function
```

328

328

5

21.866666666666667

21.866666666666667

Printing Formatted Code

We often find that we want to format the output of our code in a nice way. This includes printing string and code output. In Python, there are multiple ways to do this. Two options are listed below. Consider our `prime50_array` for this exercise.

```
In [33]: prime50_array
```

```
Out[33]: array([ 2,  3,  5,  7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47])
```

```
In [34]: #Option 1: print string and code seperated by a comma
print("array:", prime50_array)

print("max:", np.max(prime50_array)) #we can apply functions directly to
code output

#Option 2: print string and code by converting everything to a string and
concatenating
print("Maximum prime under 50 is " + str(np.max(prime50_array)))
```

```
array: [ 2  3  5  7 11 13 17 19 23 29 31 37 41 43 47]
max: 47
Maximum prime under 50 is 47
```



Code Comprehension:

Take the last output and add a period in the code output.

```
In [35]: print("Maximum prime under 50 is " + str(np.max(prime50_array)) + ".")
Maximum prime under 50 is 47.
```

We can easily create arrays by specifying a range.

Calling `np.arange()` creates a half-closed interval `[start,end)` where the end value is not included.

```
In [36]: np.arange(4,10)
```

```
Out[36]: array([4, 5, 6, 7, 8, 9])
```

```
In [37]: #if you leave out the start, the default is zero
print(np.arange(10))
```

```
[0 1 2 3 4 5 6 7 8 9]
```

We can specify a step size we want to increment by. If we leave out the step, the default is one.

```
In [38]: print(np.arange(1,31,2))
```

```
[ 1  3  5  7  9 11 13 15 17 19 21 23 25 27 29]
```

Another reason why arrays are useful!

Elementwise operations!

```
In [39]: array_1 = np.arange(10)
array_2 = np.array([1,2,3,4,5,6,7,8,9,10])
difference_array = array_1 - array_2
difference_array
```

```
Out[39]: array([-1, -1, -1, -1, -1, -1, -1, -1, -1, -1])
```



Code comprehension: Multiple Choice

What will be printed?

```
import numpy as np
a = np.array([1,2,3,5,8])
b = np.array([0,3,4,2,1])
c = a + b
c = c*a
print (c[2])
```

A. 7

B. 12

C. 10

D. 21

E. 28



Code comprehension: Multiple Choice

What will be output for the following code?

```
number_array = np.array([1,2,3,5,8])
number_array = number_array + 1
print(number_array[1])
```

A. 0

B. 1

C. 2

D. 3

We may have higher dimensional arrays.

Organized into rows and columns

```
In [40]: arr_2d = np.array([[1, 2, 3], [4, 5, 6]]) #specify first row then second
print(arr_2d)
```

```
[[1 2 3]
 [4 5 6]]
```

```
In [41]: arr_2d.shape #outputs (#rows, #columns)
```

```
Out[41]: (2, 3)
```

We can reshape our arrays

```
In [42]: reshaped_arr = np.reshape(arr_2d, 6)
```

```
reshaped_arr.shape
```

```
Out[42]: (6,)
```

```
In [43]: np.reshape(arr_2d, (3,-1)) # the unspecified value is inferred to be 2  
np.reshape(arr_2d, (3,-1)).shape
```

```
Out[43]: (3, 2)
```

```
In [44]: arr_2d[:,2] #all of the rows, column at index 2
```

```
Out[44]: array([3, 6])
```

```
In [45]: arr_2d[0,1] #element with row index 0 and column index 1
```

```
Out[45]: 2
```



Working with Arrays: Exercise

Use the following array to answer the questions

```
In [46]: random_number_array = np.array([32, 56, 78, 3, 15, 109, 13, 24, 58, 61,  
90, 93, 45, 21, 46])
```

1. Remove elements 2 and 3

```
In [47]: np.delete(random_number_array, [2, 3])
```

```
Out[47]: array([ 32,  56,  15, 109,  13,  24,  58,  61,  90,  93,  45,  21,  46])
```

2. Use a method to find the minimum value in the array

```
In [48]: random_number_array.min()
```

```
Out[48]: 3
```

3. Find the 4th smallest element in the array

```
In [49]: np.sort(random_number_array)[3]
```

```
Out[49]: 21
```

4. Create an array with 60 elements that corresponds to the 60 minutes in an hour.

```
In [50]: minutes = np.arange(60)
minutes
```

```
Out[50]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
        51, 52, 53, 54, 55, 56, 57, 58, 59])
```

5. Starting (and including) minute 4, extract every 5th element.

```
In [51]: minutes[4::5]
```

```
Out[51]: array([ 4,  9, 14, 19, 24, 29, 34, 39, 44, 49, 54, 59])
```

Another Built-in Collection of Data: Dictionaries

A **dictionary** is a set of "key: value" pairs where each key is unique.

We can create a dictionary with curly brackets "{}"

Entries of a dictionary are of the form "key: value"

```
In [52]: survey_dict = {0: "Strongly Disagree", 1: "Disagree", 2: "No opinion", 3:
"Agree", 4: "Strongly Agree"}
```

We can access values of a dictionary by their key. (This is why all keys must be unique!)

```
In [53]: survey_dict[1]
```

```
Out[53]: 'Disagree'
```

Dictionaries are useful for storing and extracting data! Think of them like an address book where you look up someone's address by finding their name.

A few useful operations with dictionaries:

- Add an entry
- Delete an entry

Add and delete pairs!

```
In [54]: del survey_dict[1]
```

```
In [55]: survey_dict
```

```
Out[55]: {0: 'Strongly Disagree', 2: 'No opinion', 3: 'Agree', 4: 'Strongly Agree'}
```

```
In [56]: #this adds a key-value pair  
survey_dict['new_key'] = 'new value'
```

```
In [57]: survey_dict
```

```
Out[57]: {0: 'Strongly Disagree',  
          2: 'No opinion',  
          3: 'Agree',  
          4: 'Strongly Agree',  
          'new_key': 'new value'}
```

Note we can also determine if keys are contained in the dictionary

```
In [58]: 3 in survey_dict
```

```
Out[58]: True
```

```
In [59]: "Disagree" in survey_dict
```

```
Out[59]: False
```

```
In [60]: 'new_key' in survey_dict
```

```
Out[60]: True
```

Keep in mind, keys do not all need to be the same type, although it may make more sense to keep them that way.

```
In [61]: survey_dict_2 = {"Strongly Disagree": 0 , "Disagree": 1 , 2: "No opinion",  
                          3: "Agree", 4: "Strongly Agree"}
```

```
In [62]: survey_dict_2["Disagree"]
```

```
Out[62]: 1
```

```
In [63]: list(survey_dict_2)
```

```
Out[63]: ['Strongly Disagree', 'Disagree', 2, 3, 4]
```

Dictionary Methods

There are methods that you can use on dictionaries, just like with lists and arrays. Here are a few.

```
In [64]: print(dir(survey_dict_2))
```



```
[ '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
```

```
In [65]: survey_dict.keys()
```

```
Out[65]: dict_keys([0, 2, 3, 4, 'new_key'])
```

```
In [66]: survey_dict
```

```
Out[66]: {0: 'Strongly Disagree',
          2: 'No opinion',
          3: 'Agree',
          4: 'Strongly Agree',
          'new_key': 'new value'}
```



Working with Dictionaries: Short Exercise

Below is a dictionary containing total number of homicides in the United States in 2021, by state. (Published by Statista Research Department, Oct 14, 2022). Note Washington D.C. is included as 'District of Columbia'.

1. Pick a few states of interest and find their homicide number (use code here...do not just visually search the dictionary!)
2. Find the number of keys in the dictionary. Does this imply all 50 states are included here?
3. What are some limitations to this data?

```
In [1]: homicide_dict = {'Texas': 2064, 'North Carolina': 928, 'Ohio': 824,
                        'Michigan': 747, 'Georgia': 728, 'Tennessee': 672, 'Missouri': 593,
                        'Virginia': 562, 'South Carolina': 548, 'Illinois': 514, 'Pennsylvania':
                        510, 'Louisiana': 447, 'Indiana': 438, 'Alabama': 370, 'Kentucky': 365,
                        'Colorado': 358, 'Washington': 325, 'Arkansas': 321, 'Wisconsin': 315,
                        'Oklahoma': 284, 'Nevada': 232, 'Minnesota': 203, 'Arizona': 190,
                        'Oregon': 188, 'New Mexico': 169, 'Mississippi': 149, 'Connecticut': 148,
                        'Maryland': 138, 'New Jersey': 137, 'Massachusetts': 132, 'New York': 124,
                        'California': 123, 'District of Columbia': 109, 'West Virginia': 95,
                        'Delaware': 94, 'Kansas': 87, 'Utah': 85, 'Iowa': 70, 'Rhode Island': 38,
                        'Idaho': 36, 'Montana': 31, 'South Dakota': 26, 'Nebraska': 25, 'Alaska':
                        18, 'Maine': 18, 'Wyoming': 17, 'New Hampshire': 14, 'North Dakota': 14,
                        'Vermont': 8, 'Hawaii': 6}
```

```
In [2]: # 1. Homicide numbers for a few states of interest
        print("Illinois:", homicide_dict["Illinois"])
```

```
print("California:", homicide_dict["California"])
print("Texas:", homicide_dict["Texas"])
```

```
Illinois: 514
California: 123
Texas: 2064
```

```
In [3]: # 2. Number of keys
print("Number of keys:", len(homicide_dict))
```

```
Number of keys: 50
```

There are 50 keys, but one is "District of Columbia", which is not a state. So this does not include all 50 states (it is 49 states plus D.C.).

3. Limitations: these are raw totals, not rates per population, so larger states look worse by count alone. It is also only one year (2021) and gives no context on causes or differences in how states report.



Reflection

In the following markdown cell, write about something you learned while working through this notebook.

In this notebook I learned the differences between lists, NumPy arrays, and dictionaries. Slicing was the most helpful idea for me, especially using a step value to pull out every nth element. I also liked that dictionaries let me look up a value by its key instead of searching through the data by hand.